In [ ]:
```
#Python step slicing
```

In [1]:
```
s1 = 'welcome'
s1
```

Out[1]:
```
'welcome'
```

In [2]:
```
s1[1:6:2]    #step slicing means..1 is start point, 6 is end point and 2 is steps t
```

Out[2]:
```
'ecm'
```

In [4]:
```
s1[::3]   #:: means all. w will print and slice by counting 3 everytime
```

Out[4]:
```
'wce'
```

Python typecasting

In [ ]:
```
car()
```

In [ ]:
```
asian_Cars('Tata','Mahindra')
```

In [ ]:
```
german_cars('Audi,'BMW','Ferrari','Vw')
```

In [5]:
```
#typecasting all datat types to integer
```

In [6]:
```
int(2.3) #float to int
```

Out[6]:
```
2
```

In [7]:
```
int(2.3,4.1)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[7], line 1
----> 1 int(2.3,4.1)

TypeError: 'float' object cannot be interpreted as an integer
```

In [8]:
```
int(True)  # bool to int
```

Out[8]:
```
1
```

In [9]:
```
int(1+2j)   #complex to int is not possible
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[9], line 1
----> 1 int(1+2j)

TypeError: int() argument must be a string, a bytes-like object or a real number, no
t 'complex'
```

In [10]:
```
int('10') #string to int conversion
```

Out[10]:
```
10
```

In [11]:
```
int('ten')  #text string is not casted to int and only number string is possible
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[11], line 1
----> 1 int(      )

ValueError: invalid literal for int() with base 10: 'ten'
```

In [ ]:
```
# diff data types to float
```

In [12]:
```
float (20)
```

Out[12]:
```
20.0
```

In [13]:
```
float(True)
```

Out[13]:
```
1.0
```

In [14]:
```
float(False)
```

Out[14]:
```
0.0
```

In [15]:
```
float('10')
```

Out[15]:
```
10.0
```

In [16]:
```
float('ten')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[16], line 1
----> 1 float(      )

ValueError: could not convert string to float: 'ten'
```

In [17]:
```
float(2+3j)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[17], line 1
----> 1 float(2+3j)

TypeError: float() argument must be a string or a real number, not 'complex'
```

In [ ]:
```
# All other data types are possible except text string and complex
```

In [ ]:
```
# Non Zero elements are always true in boolean
```

In [18]:
```
bool(2)
```

Out[18]:
```
True
```

In [19]:
```
bool(2.3)
```

Out[19]:
```
True
```

In [20]:
```
bool(1+3j)
```

Out[20]:
```
True
```

In [21]:
```
bool(0)
```

Out[21]:
```
False
```

In [22]:
```
bool('ten')
```

Out[22]:
```
True
```

In [23]:
```
bool()
```

Out[23]:
```
False
```

In [ ]:
```
# to get size of byte storage
```

In [24]:
```
val1 =['10', '20']
```

In [25]:
```
import sys
sys. getsizeof(val1)
```

Out[25]:
```
72
```

In [ ]:
```
_init_  #python constructor will be doing this byte storage calculation which is in
```

In [ ]:
```
#complex datatype typecasting
```

In [26]:
```
complex(12)  #complex contains one real and on imaginary arguments
```

Out[26]:
```
(12+0j)
```

In [27]:
```
complex(12.3)
```

Out[27]:
```
(12.3+0j)
```

In [28]:
```
complex(10,20)
```

Out[28]:
```
(10+20j)
```

In [29]:
```
complex(12,20,23)   #it cannot take more than 2 arguments
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[29], line 1
----> 1 complex(12,20,23)

TypeError: complex() takes at most 2 arguments (3 given)
```

In [30]:
```
complex(True)
```

Out[30]:
```
(1+0j)
```

In [31]:
```
complex(False)
```

Out[31]:
```
0j
```

In [32]:
```
complex(True, False)
```

Out[32]:
```
(1+0j)
```

In [33]:
```
complex('10')
```

Out[33]:
```
(10+0j)
```

In [34]:
```
complex('10' , '20')   # the second argument in complex cant be a string
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[34], line 1
----> 1 complex(      ,      )

TypeError: complex() can't take second arg if first is a string
```

In [35]:
```
str(100)
```

Out[35]:
```
'100'
```

In [36]:
```
str(2.3)
```

Out[36]:
```
'2.3'
```

In [37]:
```
str(True)
```

Out[37]:
```
'True'
```

In [38]:
```
str(False)
```

Out[38]:
```
'False'
```

In [39]:
```
str(1+2j)
```

Out[39]:
```
'(1+2j)'
```

In [40]:
```
str(3,2)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[40], line 1
----> 1 str(3,2)

TypeError: str() argument 'encoding' must be str, not int
```

In [ ]:
```
Implicit typecasting means automatic typecasting occurs where as manual oprators
```

Python operators

Arithmetic operators

In [41]:
```
x1,y1 =10,5
```

In [42]:
```
x1+y1
```

Out[42]:
```
15
```

In [43]:
```
x1-y1
```

Out[43]:
```
5
```

In [44]:
```
x1*y1
```

Out[44]:
```
50
```

In [45]:
```
x1/y1
```

Out[45]:
```
2.0
```

In [47]:
```
x1%y1
```

Out[47]:
```
0
```

In [46]:
```
x1//y1
```

Out[46]:
```
2
```

In [48]:
```
3*2
```

Out[48]: 6

In [49]:
```
3**2 #if we have 2 multiplication symbols it means that it works as 3 power of 2
```

Out[49]: 9

Assignment operators

In [16]:
```
x = 3
x
```

Out[16]: 3

In [17]:
```
x = x + 3
x
```

Out[17]: 6

In [18]:
```
x += 2
x
```

Out[18]: 8

In [23]:
```
y = 3
```

In [24]:
```
y *= 3
y
```

Out[24]: 9

In [26]:
```
y *= 2
y
```

Out[26]: 36

In [29]:
```
x //= 2
x
```

Out[29]: 18

Relational operators

In [37]:
```
a = 5
b = 6
```

In [38]:
```
a < b
```

Out[38]: True

In [39]:
```
b < a
```

Out[39]: False

In [41]:
```
a == b
```

Out[41]: True

In [42]:
```
a != b
```

Out[42]: False

In [43]:
```
print(a)
```

```
6
```

In [44]:
```
a <= b
```

Out[44]: True

Logical operators

In [ ]:
```
# It works according to the truth table AND gate OR gate
```

In [ ]:
```
a = 5
b = 3
```

In [46]:
```
a < 8 and b < 5
```

Out[46]: False

In [47]:
```
a < 8 or b < 5
```

Out[47]: True

In [48]:
```
x = True
x
```

Out[48]: True

In [50]:
```
not x
```

Out[50]: False

unary operators

In [ ]:
```
#we use only one operator called '-' also called nagtion
```

In [30]:
```
n=2
n
```

Out[30]: 2

In [32]:
```
m=-n
m
```

Out[32]: -2

In [ ]:
```
negation the minus cannot be stored in the system as per to ASCII table
```

Bitwise operators

In [ ]:
```
# These operators works like logical operators but related to storage in bits
```

In [ ]:
```
#Binary number system , octal number system (0 -7), decimal number system (0-9) , h
```

number system

In [51]:
```
25
```

Out[51]: 25

In [52]:
```
bin(25)  #convert no to binary no
```

Out[52]: '0b11001'

In [53]:
```
int(0b11001) #converting bin no to number system by multiplying every no with 2^0 ,
```

Out[53]: 25

In [54]:
```
oct(25)  # doing div with 8^0 and 8^1
```

Out[54]: '0o31'

In [55]:
```
int(0o31)
```

Out[55]: 25

In [ ]:
```
#decimal
```

In [56]:
```
hex(16) # doing div with 16
```

Out[56]: '0x10'

In [57]:
```
print(0xa)
print(0xb)
print(0xc)
print(0xd)
print(0xe)
print(0xf)
```

```
10
11
12
13
14
```

In [ ]:
```
# number system used for bitwise operator
(COMPLEMENT ~ , &,^ XOR, <<,>>)
```

In [59]:
```
a = 2
-a
```

Out[59]:
```
-2
```

In [4]:
```
~10
```

Out[4]:
```
-11
```

In [5]:
```
#instead of AND OR we directly use & symbol
```

In [66]:
```
12 & 13   #here we need to convert numbers to binary digits and apply 'AND' 'OR' lo
```

Out[66]:
```
12
```

In [69]:
```
12 & 20  #Bitwise AND
```

Out[69]:
```
4
```

In [70]:
```
12 | 13 # BITWISE OR
```

Out[70]:
```
13
```

In [71]:
```
12 ^ 13  # XOR  if same items we get 0 and if different no we get 1
```

Out[71]:
```
1
```

In [72]:
```
25 ^ 30
```

Out[72]:
```
7
```

In [ ]:
```
#<< in left shift we need to add extra zeroes to the binary no of the value
```

In [73]:
```
12 << 1
```

Out[73]:
```
24
```

In [74]:
```
12 >> 1
#>> right shift means we loose a bit on the right side
```

Out[74]: 6

In [75]:
```
13 >> 3
```

Out[75]: 1

In [77]:
```
#Print format method function
# Flower braces are automatically understood by python based on mentioning them in
```

In [78]:
```
num1=25
num2=30
add=num1+num2
print(add)
```

```
55
```

In [80]:
```
num1=25
num2=30
add=num1+num2
print('The addition of {} and {} is = {}' .format(num1,num2,add))
```

```
The addition of 25 and 30 is = 55
```

In [82]:
```
num1=25
num2=30
add=num1-num2
print('The subtraction of {} and {} is = {}' .format(num1,num2,add))
```

```
The subtraction of 25 and 30 is = -5
```

In [ ]:
```
# f string method
# it means the variable must be called inside flower braces
```

In [84]:
```
num4=30
num5=15
add=num4+num5
print(f'The addition of {num4} and {num5} is = {add}')
```

```
The addition of 30 and 15 is = 45
```

In [ ]:
```
#print funtion with end statement
# this is generally used to excute multiple stmts in single line without creating n
```

In [87]:
```
print('Hello' ,end =' ')  # 1st stmt
print('Gud Afternoon')  # 2nd stmt
```

```
Hello Gud Afternoon
```

keynotes IP Address are written in Hexa Decimal Format

In [2]:
```
import math
math.sqrt(25)
```

Out[2]:
```
5.0
```

In [3]:
```
round(math.sqrt(25))
```

Out[3]:
```
5
```

In [8]:
```
import keyword
keyword.kwlist
```

Out[8]:
```
['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
 'class',
 'continue',
 'def',
 'del',
 'elif',
 'else',
 'except',
 'finally',
 'for',
 'from',
 'global',
 'if',
 'import',
 'in',
 'is',
 'lambda',
 'nonlocal',
 'not',
 'or',
 'pass',
 'raise',
 'return',
 'try',
 'while',
 'with',
 'yield']
```

In [5]:
```
x = round(math.sqrt(25))
x
```

Out[5]:
```
5
```

In [6]:
```
math.floor(2.7)
```

Out[6]: 2

In [7]:
```
math.ceil(2.7)
```

Out[7]: 3

In [8]:
```
pow(4,3)
```

Out[8]: 64

In [9]:
```
math.pi
```

Out[9]: 3.141592653589793

In [11]:
```
pi=3.172  #In python we dont have constant values , we only have variables
pi
```

Out[11]: 3.172

In [12]:
```
help()
```

```
Welcome to Python 3.13's help utility! If this is your first time using
Python, you should definitely check out the tutorial at
https://docs.python.org/3.13/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules.  To get a list of available
modules, keywords, symbols, or topics, enter "modules", "keywords",
"symbols", or "topics".

Each module also comes with a one-line summary of what it does; to list
the modules whose name or summary contain a given string such as "spam",
enter "modules spam".

To quit this help utility and return to the interpreter,
enter "q", "quit" or "exit".

You are now leaving help and returning to the Python interpreter.
If you want to ask for help on a particular object directly from the
interpreter, you can type "help(object)".  Executing "help('string')"
has the same effect as typing a particular string at the help> prompt.
```

In [14]:
```
import math as m
```

In [23]:
```
from math import sqrt, pow
print(m.pow(2,4))
print(m.sqrt(9))
```

```
16.0
3.0
```

In [24]:
```
print(math.e) #epsilon values
```

```
2.718281828459045
```

In [25]:
```
print(e)
```

```
2.718281828459045
```

In [26]:
```
from math import *
```

Input functions

In [27]:
```
input()
```

Out[27]:
```
'hi'
```

In [28]:
```
a=input()   #we need to mention typecasting otherwise it will consider it as string
b=input()   #press shift tab to read decsription about the functions
c=a+b
print(c)
```

```
12
```

In [29]:
```
a=int(input())
b=int(input())
c=a+b
print(c)
```

```
3
```

In [33]:
```
x = input('Enter username: ')
y = input('Enter password: ')
z = x + y
print(z)
```

```
moulyabubf
```

In [34]:
```
str = input('enter  a name: ')
print(str)
```

```
moulya
```

In [37]:
```
str = input('Enter name and get specific chars: ') [2]
print(str)
```

```
o
```

In [39]:
```
str = input('Enter name: ') [1:3]  # slicing rule is start number is included but s
print(str)
```

```
ou
```

In [ ]:

In [ ]: